GESTIONE PREVENTIVI

Documentazione Tecnica

Frontend + Backend — Versione 1.0

Frontend	HTML · CSS · JavaScript
Backend	Node.js · Express · SQLite
Real-time	Socket.IO
Database	SQLite (Preventivi.db)

1. Panoramica dell'Applicazione

Gestione Preventivi è un sistema web completo per la gestione dei preventivi di una concessionaria o officina moto. Permette di tenere traccia dei clienti, dei preventivi emessi, delle marche e dei modelli di veicolo, con aggiornamenti in tempo reale tra più utenti.

Il sistema è diviso in due parti:

- Frontend: le pagine HTML/CSS/JS con cui interagisce l'utente
- Backend: il server Node.js che gestisce i dati e le API REST

Accesso al sistema

Campo	Valore
URL	http://IP-SERVER:3000
Utente default	Admin
Password default	Admin123!
Health check	http://IP-SERVER:3000/api/health

Struttura delle cartelle

Percorso	Contenuto
frontend/	Pagine HTML, CSS e JavaScript del client
frontend/index.html	Pagina di login
frontend/home.html	Dashboard principale
frontend/js/	Tutti gli script JavaScript
frontend/company-info.json	Dati aziendali (indirizzo, P.IVA, ecc.)
backend/server.js	Punto di ingresso del server Node.js
backend/routes/	Route API Express
backend/db/init.js	Inizializzazione database e creazione tabelle
backend/db/Preventivi.db	File del database SQLite
backend/db/Seed.js	Script opzionale per dati di esempio

2. Frontend

Pagine HTML

File	CSS	Funzione
index.html	login.css	Login con form nome/password e dati aziendali in calce
home.html	styles.css	Dashboard con sidebar e 5 sezioni navigabili

Sezioni della Dashboard

Sezione	data-section	Funzione
Clienti	clienti	Anagrafica clienti con telefono, email, data passaggio e stato ricontatto
Preventivi	ordini	Lista e gestione di tutti i preventivi con filtri avanzati e stampa
Marche	marche	CRUD marche veicoli con conteggio modelli e preventivi collegati
Modelli	modelli	CRUD modelli con ricerca marca ad autocomplete
Utenti	utenti	Gestione account con hashing bcrypt e validazione password

Script JavaScript — Ordine di caricamento (home.html)

#	File	Cosa fa
1	realtime.js	Connessione Socket.IO e gestione eventi real-time
2	config.js	Costante API_URL='/api', variabili globali (clienti[], ordini[], marche[], modelli[], utenti[]), funzioni formatDate, formatPhoneNumber
3	ui.js	showNotification (toast), showConfirmModal (promise-based)
4	company-loader.js	Carica company-info.json e inietta i dati in sidebar e stampe
5	searchable-select.js	Componente autocomplete riutilizzabile per clienti e marche nei modal
6	clienti.js	CRUD clienti: loadClienti, renderClienti, filtri con localStorage
7	preventivi.js	CRUD preventivi: loadOrdini, renderOrdini, filtri combinati
8	marche.js	CRUD marche con filtro e modal
9	modelli.js	CRUD modelli con searchable-select per la marca
10	utenti.js	CRUD utenti con toggle password e validazione
11	stampa.js	Generazione HTML di stampa per preventivi raggruppati per cliente
1	app.js	Bootstrap app: auth guard, navigazione SPA, hamburger mobile,

2		logout

Variabili di Stato Globali (config.js)

Variabile	Tipo	Descrizione
API_URL	string	Base URL '/api' per tutte le chiamate REST
clienti / allClienti	Array	Lista filtrata / lista completa clienti
ordini / allOrdini	Array	Lista filtrata / lista completa preventivi
marche / allMarche	Array	Lista filtrata / lista completa marche
modelli / allModelli	Array	Lista filtrata / lista completa modelli
utenti	Array	Lista utenti del sistema

Componente Autocomplete (searchable-select.js)

Il componente createSearchableSelect() è riutilizzato in due contesti: nella ricerca del cliente nel modal preventivo, e nella ricerca della marca nel modal modello. Ogni istanza è indipendente e ha una propria API.

Istanza globale	Usata in	Ricerca
clienteSearchOrdine	Modal Nuovo/Modifica Preventivo	Nome, telefono o email del cliente
marcaModelloSearchOrdine	Modal Nuovo/Modifica Preventivo	Nome marca o modello (campo unificato)
marcaSearchModello	Modal Nuovo/Modifica Modello	Nome della marca

Quando si seleziona un modello nel preventivo, il componente recupera automaticamente l'ID della marca collegata e lo salva nel campo nascosto ordineMarca, che viene poi inviato al backend.

Sistema di Filtri (persistenza localStorage)

Ogni sezione salva il termine di ricerca corrente in localStorage. Al ritorno sulla sezione il filtro viene ripristinato automaticamente.

Sezione	Chiave localStorage
Clienti	filter_clienti_search
Preventivi (testo)	filter_ordini_search
Marche	filter_marche_search
Modelli	filter_modelli_search
Utenti	filter_utenti_search

3. Backend

Server (server.js)

Il server è scritto in Node.js con Express. Serve sia le API REST che i file statici del frontend e gestisce Socket.IO per gli aggiornamenti in tempo reale. Ogni richiesta API viene loggata in console con timestamp, metodo HTTP e IP del client.

- Porta: 3000 (configurabile con variabile d'ambiente PORT)
- Avvio: node server.js oppure pm2 start server.js
- Il frontend è servito dalla cartella ../frontend
- Tutte le route non-API restituiscono index.html (SPA routing)

Route API

Prefisso	File	Gestisce
/api/auth	routes/auth.js	Login con verifica bcrypt
/api/clienti	routes/clienti.js	CRUD clienti con blocco eliminazione se ha preventivi
/api/ordini	routes/ordini.js	CRUD preventivi con JOIN a clienti, modelli e marche
/api/marche	routes/marche.js	CRUD marche con blocco eliminazione se ha modelli o preventivi
/api/modelli	routes/modelli.js	CRUD modelli con blocco eliminazione se ha preventivi
/api/utenti	routes/utenti.js	CRUD utenti con hashing bcrypt e validazione password
/api/health	server.js	Health check: stato, uptime, IP pubblico, connessioni socket

Endpoint API completi

Metodo	Endpoint	Funzione
POST	/api/auth/login	Login: cerca utente per nome, confronta password con bcrypt
GET	/api/clienti	Lista clienti con conteggio preventivi collegati (LEFT JOIN ordini)
GET	/api/clienti/:id	Dettaglio cliente con array di tutti i suoi preventivi
POST	/api/clienti	Crea cliente (nome univoco obbligatorio)
PUT	/api/clienti/:id	Aggiorna cliente (mantiene i campi non passati)
DELETE	/api/clienti/:id	Elimina cliente (bloccato se ha preventivi)
GET	/api/ordini	Lista preventivi con JOIN a clienti, modelli, marche — ordinati per data DESC

GET	/api/ordini/cliente/:id	Preventivi di un cliente specifico
POST	/api/ordini	Crea preventivo (cliente obbligatorio, modello/marca facoltativi)
PUT	/api/ordini/:id	Aggiorna preventivo
DELETE	/api/ordini/:id	Elimina preventivo
GET	/api/marche	Lista marche con conteggio modelli e preventivi collegati
POST	/api/marche	Crea marca (nome univoco)
PUT	/api/marche/:id	Aggiorna nome marca
DELETE	/api/marche/:id	Elimina marca (bloccata se ha modelli o preventivi)
GET	/api/modelli	Lista modelli con nome marca e conteggio preventivi
POST	/api/modelli	Crea modello collegato a una marca
PUT	/api/modelli/:id	Aggiorna modello
DELETE	/api/modelli/:id	Elimina modello (bloccato se ha preventivi)
GET	/api/utenti	Lista utenti (id, nome, created_at — senza password)
POST	/api/utenti	Crea utente con password hashata bcrypt
PUT	/api/utenti/:id	Aggiorna nome e/o password
DELETE	/api/utenti/:id	Elimina utente (bloccato se è l'ultimo)

4. Database (SQLite)

Il database è un file SQLite salvato in backend/db/Preventivi.db. Viene creato automaticamente al primo avvio con tutte le tabelle e un utente Admin di default (Admin / Admin123!). Le foreign key sono abilitate con PRAGMA foreign_keys = ON.

Tabelle

Tabella	Colonne principali	Funzione
utenti	id, nome (UNIQUE), password, created_at	Account di accesso con password hashata bcrypt
marche	id, nome (UNIQUE), created_at	Case produttrici dei veicoli
clienti	id, nome (UNIQUE), num_tel, email, data_passaggio, flag_ricontatto, created_at	Anagrafica clienti con stato ricontatto
modelli	id, nome (UNIQUE), marche_id (FK), created_at	Modelli di veicolo collegati a una marca
ordini	id, cliente_id (FK), data_movimento, modello_id (FK), marca_id (FK), note, contratto_finito, created_at	Preventivi collegati a cliente, modello e marca

Relazioni tra tabelle

Tabella	Chiave esterna	Punta a
modelli	marche_id	marche.id
ordini	cliente_id	clienti.id
ordini	modello_id	modelli.id
ordini	marca_id	marche.id

Campo contratto_finito

La colonna contratto_finito nella tabella ordini è stata aggiunta con una migrazione sicura (ALTER TABLE). Il backend esegue automaticamente l'ALTER TABLE all'avvio: se la colonna esiste già, l'errore viene intercettato e ignorato. I record esistenti ricevono il valore 0 (non concluso) per default.

Campo flag_ricontatto nei clienti

Analogamente, flag_ricontatto e data_passaggio sono stati aggiunti con migrazioni sicure alla tabella clienti. Il valore 0 significa Da ricontattare, il valore 1 significa Ricontattato. Il frontend mostra badge colorati corrispondenti.

Regole di integrità

- Non si può eliminare una marca se ha modelli o preventivi collegati
- Non si può eliminare un modello se ha preventivi collegati
- Non si può eliminare un cliente se ha preventivi collegati

- Non si può eliminare l'ultimo utente rimasto nel sistema
- Nome cliente, nome marca, nome modello e nome utente devono essere univoci (UNIQUE)

5. Aggiornamenti Real-Time (Socket.IO)

Il server usa Socket.IO per notificare tutti i client connessi quando i dati cambiano. Questo permette a più utenti di lavorare contemporaneamente senza dover ricaricare la pagina.

Pattern di funzionamento

- Il client si connette al server Socket.IO all'avvio (realtime.js)
- Dopo ogni operazione CRUD il backend emette un evento specifico tramite io.emit()
- Il frontend ascolta gli eventi e ricarica i dati della sezione corrispondente
- La connessione usa WebSocket con fallback su polling HTTP

Eventi Socket emessi dal backend

Evento	Emesso da	Azione frontend
cliente_aggiunto / cliente_modificato / cliente_eliminato	routes/clienti.js	loadClienti()
clienti_aggiornati	routes/clienti.js	loadClienti()
ordine_aggiunto / ordine_modificato / ordine_eliminato	routes/ordini.js	loadOrdini()
ordini_aggiornati	routes/ordini.js	loadOrdini()
marca_eliminata / marche_aggiornate	routes/marche.js	loadMarche()
modello_eliminato / modelli_aggiornati	routes/modelli.js	loadModelli()
utente_aggiunto / utente_modificato / utente_eliminato	routes/utenti.js	loadUtenti()
utenti_aggiornati	routes/utenti.js	loadUtenti()

Configurazione Socket.IO

- pingTimeout: 60000 ms — tempo massimo senza risposta prima di considerare il client disconnesso
- pingInterval: 25000 ms — frequenza dei ping di controllo connessione
- transports: ['websocket', 'polling'] — WebSocket preferito, polling come fallback
- CORS: origin '*' — accetta connessioni da qualsiasi origine (restringere in produzione)

6. Sistema di Stampa (stampa.js)

I preventivi vengono stampati come documento HTML in un iframe invisibile, inviato direttamente alla stampante del browser. La stampa rispetta i filtri attivi: vengono stampati solo i preventivi visibili in quel momento.

Struttura del documento di stampa

- Intestazione: logo aziendale centrato, nome azienda, indirizzo, P.IVA, telefono, email
- Corpo: preventivi raggruppati per cliente, ordinati alfabeticamente per nome cliente
- Per ogni cliente: nome, cellulare, email, data passaggio, stato ricontatto, numero totale preventivi
- Per ogni preventivo del cliente: data, marca, modello, stato contratto (badge colorato), note
- Piè di pagina: data e ora di generazione del documento

Flusso di stampa

- printOrdiniDiretta() verifica che ci siano preventivi da stampare
- Carica i dati aziendali da company-info.json (con cache in memoria)
- Raggruppa i preventivi per cliente_id con groupOrdiniByCliente()
- Genera l'HTML completo con generatePrintDocumentOrdiniPerCliente()
- Crea un iframe invisibile, scrive l'HTML e chiama contentWindow.print()
- Rimuove l'iframe dopo 1 secondo dalla chiusura del dialog di stampa

Dati aziendali nella stampa

I dati aziendali vengono letti dal file company-info.json che si trova nella cartella frontend. Per aggiornare il logo, l'indirizzo o la P.IVA nelle stampe è sufficiente modificare questo file JSON, senza toccare il codice.

7. Installazione e Avvio

Requisiti

- Node.js versione 16 o superiore
- NPM (incluso con Node.js)
- Server Linux (VPS) o Windows con porta 3000 aperta

Installazione

#	Comando	Descrizione
1	cd backend	Entra nella cartella backend
2	npm install	Installa le dipendenze
3	node server.js	Avvia il server
4	(opzionale) node db/Seed.js	Popola il DB con dati di esempio

Avvio in produzione con PM2

- npm install -g pm2
- pm2 start server.js --name preventivi
- pm2 save
- pm2 startup

Dipendenze Node.js

Pacchetto	Funzione
express	Framework web per le API REST
socket.io	Comunicazione real-time WebSocket
bcrypt	Hashing sicuro delle password utente
sqlite3	Driver per il database SQLite
cors	Gestione policy CORS per richieste cross-origin

8. Eliminazione Multipla (aggiornamento)

Aggiornamento della documentazione tecnica di Gestione Preventivi: introduce la selezione multipla con eliminazione di più righe in un'unica richiesta. Per ogni elemento selezionato il server esegue la stessa logica dell'eliminazione singola; gli elementi non eliminabili vengono saltati e restituiti con il motivo, senza interrompere l'operazione.

Endpoint API

Metodo e URL	Body	Note
POST /api/clienti/elimina-multiple	{ "ids": [..] }	Clienti
POST /api/ordini/elimina-multiple	{ "ids": [..] }	Preventivi
POST /api/marche/elimina-multiple	{ "ids": [..] }	Marche
POST /api/modelli/elimina-multiple	{ "ids": [..] }	Modelli
POST /api/utenti/elimina-multiple	{ "ids": [..], "current_user": "nome" }	Utenti. Salta l'utente corrente

Formato della risposta

{ "success": true, "eliminati": [1, 2], "saltati": [{ "id": 3, "nome": "Honda", "motivo": "..." }] }

Una richiesta con elenco vuoto restituisce errore 400 ("Nessun elemento selezionato"). Le richieste vengono eseguite in sequenza, in modo che i controlli (ad esempio l'ultimo utente rimasto) restino corretti a ogni passo.

Regole di eliminazione

Sezione	Un elemento viene saltato quando…
Clienti	ha preventivi collegati
Preventivi	mai: si possono sempre eliminare
Marche	ha modelli o preventivi collegati
Modelli	è usato in almeno un preventivo
Utenti	è l'utente corrente oppure l'ultimo utente rimasto

Backend

Il modulo backend/src/utils/bulk.js espone eliminaMultipli(ids, fn, etichette): richiama fn(id) per ogni id, raccoglie gli eliminati e i saltati (con il messaggio d'errore come motivo). Ogni service aggiunge il metodo eliminaMultiple che riutilizza elimina, quindi eventi real-time e controlli restano quelli già esistenti.

Frontend

Il file frontend/js/crud/bulk.js gestisce checkbox di riga, casella "seleziona tutti" (con stato indeterminato), barra con contatore, conferma ed elenco finale degli elementi saltati. Le tabelle chiamano bulkCheckboxCell per ogni riga e bulkAfterRender a fine render; la selezione si azzera a ogni ricaricamento dei dati.

File aggiunti e modificati

- backend/src/utils/bulk.js (nuovo)
- frontend/js/crud/bulk.js (nuovo)
- Service, controller e routes di clienti, marche, modelli, ordini e utenti: metodo eliminaMultiple e rotta POST /elimina-multiple
- frontend/home.html, js/crud/clienti/render.js, preventivi/render.js, marche.js, modelli.js, utenti.js: colonna checkbox e barra di selezione
- frontend/styles.css: stili .bulk-col e .bulk-toolbar
